

ML Prediction Competition Report:

Predicting Percent Proficient in New York State Schools

Mohammed Saadman Chowdhury and Xuejian Wang

CUNY Baruch College

May 2026

Abstract

This report pertains to work regarding an ML prediction competition based on predicting the percentage of proficient students for New York State school assessment records. This competition was organized by Professor Michael Weylandt as part of the course STA 9890 Statistical Machine Learning. The model performance metric in question was based on MSE. The data had a highly stratified structure owing to the complex interconnected system of New York State schooling systems. Although the target/response called `PERCENT_PROFICIENT` was expressed as a numeric outcome, the data preserved important structure, whereby the observations were linked by various relationships such as school, assessment, subgroup, district, and student-count relationships. Our overall modeling process combined supervised-learning methods with structurally aware adjustments. After appropriate EDA and feature engineering, we proceeded through various model classes in increasing complexity. This started with linear models and gradually evolved towards nonlinear, residual-based, stacking, and constraint-based approaches. Feature engineering was an important procedural adjustment, especially when later in the modeling process, we decided to also test (and later successfully implement) leakage-safe target-encoding of high cardinality features due to intermediate modeling insights. Our final model presented itself to us via analysis of structural properties of the data to coalesce into an accounting-solver approach that adjusted prior predictions to improve consistency among related school-assessment-subgroup rows. The final selected submission, `submission_final_accounting_solver_convex_te_blend_prior.csv`, (private leaderboard score 30.533 MSE, public score 31.361), onto the Kaggle platform used a convex blend (stacking ensemble) of various tree-based models as a prior followed by an accounting solver-based adjustment step. Further developmental stages used complex experimental priors and matrix-factorization residual models to achieve much better private scores; however, they were not submitted in favor of interpretability and explainability. The main after-action lesson is that the modeling process successfully identified strong latent structure, but the final submission choice was overly conservative.

Introduction and Exploratory Data Analysis (EDA)

Our project was based on the STA 9890 prediction competition held in Kaggle platform, and the goal was to predict `PERCENT_PROFICIENT` for New York State school assessment records. The target showed the percentage of students who reached proficiency in tests, under a combination of school, subgroup, and assessment criteria. Our professor provided us with assessment score files (training and test data) and school-level and district-level data, and the latter two were merged with the assessment score files to form the modeling tables. The merged training table included 144,921 rows and 62 columns, while the joined test table contained 48,307 rows and 61 columns. The one-column difference was because the target variable `PERCENT_PROFICIENT` only existed in the training data.

The EDA showed that the target was bounded from 0 to 100, with obviously visible mass at both endpoints. In particular, there were a lot of zero percent proficiency or one hundred percent proficiency, with clear spikes at 0 and 100. This was not a smooth, continuous distribution, which mattered a lot because later models needed to consider this range of the response and avoided treating the outcome as an unconstrained real-valued variable. Figure 1 shows this empirical distribution.

The EDA also explored the structure of the predictors. The merged data contained numeric school and district covariates, low-cardinality categorical variables such as subgroup and assessment name, and high-cardinality identifiers including `SCHOOL`, `DISTRICT`, and `COUNTY`. Missingness was also checked after joins, especially in covariates that did not apply to every school or district type. This exploratory process inspired the workflow: converting the joined tables into model-ready feature matrices through categorical encoding, missingness handling, and later target/statistical encoding.

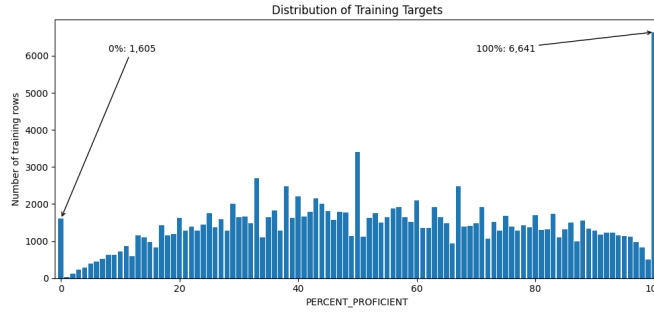


Figure 1: Target distribution

1 Data Preprocessing and Feature Engineering

The merged training and test tables were converted into numeric feature matrices after EDA. The goal of this stage was to produce consistent train and test feature representations that could support a wide range of later model classes. The pre-processing pipeline preserved the row-level assessment identifiers for diagnostics and construction of submissions; however, we removed `ASSESSMENT_ID` from the actual modeling matrix.

1.1 Base Feature Matrix

The first feature space was a base matrix built only from information available in the raw joined files. We chose not to one-hot encode the high-cardinality categorical variables `SCHOOL`, `DISTRICT`, and `COUNTY` directly, because doing so would create a sparse, high dimensional design matrix. We opted instead to replace via frequency encoding learned from the training data. Unseen test categories were assigned a frequency zero.

Missingness for numeric values was handled in two steps. For each of these specific columns, a corresponding missingness-indicator feature was created. Median imputation was used for missing numeric values from the training set, and the same training median was applied to the test set. This allowed the models to use missingness itself as a signal. In the final base pipeline, 52 missingness indicators were created.

Low-cardinality categorical variables were handled separately in terms of encoding. The features `SUBGROUP_NAME`, `ASSESSMENT_NAME`, `DISTRICT_TYPE`, and `REGION` were one-hot encoded, and the resulting training and test matrices were checked to ensure both were aligned in terms of the same columns. After all this feature engineering and removal of `ASSESSMENT_ID`, the base modeling matrices contained 162 features for both the training and test sets.

Table 1: Construction of the base feature matrix.

Step	Features	Description
Raw merged predictors	61	Joined score, school, and district predictors, excluding the target.
Frequency encoding	64	Added frequency encodings for high-cardinality identifiers.
Missingness handling	116	Added missingness indicators and median-imputed numeric variables.
Drop raw identifiers	113	Removed raw high-cardinality identifier columns.
One-hot encoding	163	Encoded low-cardinality categorical variables and aligned train/test columns.
Remove assessment ID	162	Removed <code>ASSESSMENT_ID</code> from the modeling matrix.

1.2 Target and Statistical Encoding Feature Branch

In the later stages of our workflow, we included a second branch of features based on a leakage-safe target and statistical encoding. This branch was introduced after initial models showed that school, assessment, subgroup, and geographic groupings contained strong repeated structure that was not fully captured by the base feature matrix.

For each selected grouping key, the encoding summarized historical proficiency behavior using smoothed group

statistics. The selected keys included broad groups such as assessment, subgroup, county, district, and school, as well as interaction keys such as assessment-by-subgroup, county-by-assessment, district-by-assessment, school-by-subgroup, and school-by-assessment. Because small groups can be noisy, group averages were shrunk toward the global mean using pseudo-count smoothing. More specific and sparser keys were assigned stronger smoothing.

The target/statistical encoding branch produced four features per grouping key: a smoothed mean, a student-count-weighted smoothed mean, a log-count feature, and a within-group standard-deviation feature. With 17 selected grouping keys, this added 68 engineered features. When appended to the 162-feature base matrix, the expanded feature representation contained 230 features.

To avoid target leakage, these encodings were not computed by directly averaging over the full training set for each training row. Instead, training encodings were built out-of-fold: each row received group statistics computed from other folds only. For validation or test rows, the statistics were learned from the available training rows and then applied forward. This distinction was important because target encoding can otherwise allow the model to indirectly see the response value of the row being predicted.

Table 2: Feature spaces used during modeling.

Feature space	Features	Main contents
Base feature matrix	162	Frequency encodings, missingness indicators, median-imputed numeric variables, and one-hot encoded low-cardinality categorical variables.
Expanded target/statistical encoding matrix	230	Base features plus 68 leakage-safe target/statistical encoding features from 17 grouping keys.

Validation Strategy

Two phases were involved in our validation strategy. During early exploration, we used a fixed 80/20 train-validation split with random seed 9890. This method was computationally efficient for initial screening across many feature spaces and model classes. The holdout scores were mainly for development diagnostics rather than final estimates of generalization.

After we found models that are plausible for submission, checking performance on Kaggle, we then moved to five-fold cross-validation and saved the out-of-fold prediction submission files. This was crucial to avoid leakage, which was required by response-dependent features such as target/statistical encodings. The resulting OOF files supported later model comparison, residual diagnostics, blending, stacking, and final submission reviews.

2 Model Development

Our modeling process was sequential intentionally. We started with simple models to build baselines and explore the feature space, then step forward to more complex nonlinear models, target/statistical encoding, residual correction, and finally accounting models with structural awareness and latent factors. The main value of the early models didn't come from the accuracy of their prediction, but the modeling evidence they demonstrated: signal was distributed across many predictors, nonlinear structure mattered, and later improvements required exploiting repeated school-assessment-subgroup structure.

2.1 Linear Models and Controlled Feature Expansion

The 162-feature base matrix was used for the first models. Simple linear regressions, each with only one feature, found that the best single predictor was `PERCENT_ECONOMICALLY_DISADVANTAGED`, with a validation MSE of about 581.07. However, a multiple linear regression using all base features reduced validation MSE to 312.60, showing that the prediction signal was not just focused on one dominant feature, but spread across many variables.

We then tried to expand the linear feature space in some controlled ways. First, we selected the top 10 single feature predictors for squared terms, adding 10 feature counts from 162 to 172. Second, we picked the top 5 single feature predictors for pairwise interaction terms, also adding 10 feature counts to 172. Third, both sets of additions were combined, giving a 182 feature space. The best model came from the combined squared-term and interaction space,

using backward stepwise selection, with validation MSE 308.13. Later, Ridge, Lasso, and Elastic Net showed the similar results, with validation MSE values around 308.36, 308.42, and 308.43, not improving the result. Here, Ridge means linear regression with an L_2 penalty that shrinks coefficients toward zero, Lasso uses an L_1 penalty that can set coefficients exactly to zero, and Elastic Net combines both penalties.

The next linear extension was a step-function feature space, where continuous predictors were converted into quantile-bin indicators. For example, a step-function model could evaluate different effects under different ranges of a variable, like `PERCENT_ECONOMICALLY_DISADVANTAGED`, rather than forcing that to have one constant linear effect. Our initial validation MSE was 303.57, and then we increased the predictors, bins, and resulting step features iteratively. The strongest version used 30 continuous predictors, 30 bins, and 526 added step-function features, producing a 708-feature design, which reduced the validation MSE to about 290.1. This was called the focused search in the code. Later, Ridge and Lasso refinements on this same step-function matrix only showed very tiny improvements, with the best Lasso-step model being about 290.06. This suggested that the improvement came from the expanded step-function representation rather than from sparsity. Therefore, linear models and controlled feature expansions had begun to plateau.

2.2 Tree Ensembles and Gradient Boosting

Our shift to nonlinear modelling classes marked our first major improvement, which came from tree-based models. A regression tree recursively splits the feature space into regions and predicts a constant value within each terminal leaf that is generally a mean of training observations in that leaf. This allowed the model to learn thresholds and interactions automatically for nonlinear patterns rather than relying on manually constructed squared terms or bin indicators. The best regression-tree holdout model reached a validation MSE of about 240.57, and the five-fold tree result was about 250.69.

Random forests improved further by averaging many decision trees. Each tree is trained with randomness, and the averaging reduces variance relative to a single tree. The 500-tree random forest reached OOF MSE 122.33. ExtraTrees then improved to 110.75. ExtraTrees is similar to a random forest, but it injects more randomness into how split thresholds are chosen. In our selected ExtraTrees model, `n_estimators = 250` means 250 trees were averaged, `max_features = 0.5` means each split considered only half of the features, `min_samples_leaf = 1` allowed leaves to be small, and `bootstrap = False` means each tree used the full fold-training sample rather than a bootstrap sample.

LightGBM became the strongest generic supervised model family. Unlike random forests and ExtraTrees, which train many trees independently and average them, gradient boosting builds trees sequentially: each new tree is trained to correct errors from the previous trees. The selected base-feature LightGBM used `num_leaves = 95`, `min_child_samples = 60`, `learning_rate = 0.03`, `subsample = 0.85`, `colsample_bytree = 0.90`, `reg_lambda = 5.0`, and a 12,000-tree cap. In these settings, `num_leaves` controls the complexity of each tree, `min_child_samples` prevents leaves from being based on too few rows, `learning_rate` controls how much each new tree changes the prediction, `subsample` and `colsample_bytree` randomly sample rows and columns, and `reg_lambda` applies L_2 regularization.

The selected 12,000-tree LightGBM reached OOF MSE 98.78. Because the folds were still reaching the tree cap, longer versions were trained with the same basic structure. The 80,000-tree version reached OOF MSE 94.16, and the 100,000-tree version with learning rate 0.02 reached 93.73. A weighted average of ExtraTrees and the long 100,000-tree LightGBM improved OOF MSE to 88.97 and produced a public leaderboard score of 80.662. This showed that ExtraTrees still contributed some different prediction behavior even though LightGBM was the stronger standalone model.

2.3 Target and Statistical Encoding

Our next large improvement came from the 230-feature target-encoding matrix. This feature space kept the 162 base features and added 68 leakage-safe group-statistic features. These came from 17 grouping keys, with four features per key: a smoothed group mean, a student-count-weighted smoothed mean, a group log-count, and a within-group standard deviation. The grouping keys included school, district, county, assessment, subgroup, and selected interactions among them.

A LightGBM model trained on this 230-feature representation reached OOF MSE 82.91 and public MSE 70.997. This was a much larger improvement than the earlier linear, step-function, or tree-only feature expansions. The result showed that leakage-safe target/statistical encoding made this structure usable for supervised learning.

We then blended the strongest saved prediction files using a convex weighted average. A convex blend means the model predictions are averaged with nonnegative weights that sum to one, so the blended prediction remains an average of existing predictions. The best blend placed most weight on the target/statistical-encoded LightGBM, with smaller weights on the long LightGBM models and ExtraTrees. Its OOF MSE was 78.07 and its public MSE was 69.689. This blend became the clean prior used by the final accounting-solver model in the report notebook, but it was not the universal prior for every later branch in the larger working notebook.

2.4 Other Model Families Considered

Several other model families were tested but did not become central. XGBoost and CatBoost were weaker than LightGBM on the same target/statistical-encoding feature space. Simple grouped historical means were also weak as standalone predictors; even the best `SCHOOL × ASSESSMENT_NAME` historical mean had OOF MSE 194.13, far worse than the target-encoded LightGBM. Spline and GAM-style models were tested as residual corrections after the target-encoded boosting anchor. The best spline/GAM residual correction improved OOF MSE only from 77.05 to 76.93, so the gain was too small to justify making this a main branch of exploration. These results pushed the project away from generic smooth residual corrections and toward structural relationships in the score table itself.

2.5 Accounting-Based Models

The main breakthrough came from the subgroup accounting structure. Within a fixed `SCHOOL × ASSESSMENT_NAME` group, subgroup rows were related through student counts. Since each row included `N_STUDENTS`, a predicted percentage could be converted into an estimated number of proficient students. This made it possible to use relationships such as

$$\text{All Students} \approx \text{Female} + \text{Male}$$

$$\text{All Students} \approx \text{Economically Disadvantaged} + \text{Not Economically Disadvantaged}.$$

The first accounting model directly reconstructed rows when the necessary sibling subgroup rows were available. For example, if `All Students` and `Male` were known, then `Female` could be inferred by subtraction in count space. This direct reconstruction reduced OOF MSE to 55.64 and produced a public score of 37.424, far below the previous public score of nearly 68.9. This showed that the largest missing signal was not another generic regression feature, but the internal accounting structure of the assessment records.

The next accounting model treated each `SCHOOL × ASSESSMENT_NAME` group as a small constrained system. Instead of only reconstructing rows when a direct subtraction was possible, the solver adjusted unknown subgroup counts so that they stayed close to a prior prediction while satisfying or nearly satisfying the subgroup identities. In the original working notebook, this branch used several experimental priors and fallback rules. In the cleaned final notebook, the report version uses the convex target/statistical-encoding stack as the prior. This cleaned version reached OOF MSE 54.29 and public MSE 31.361, and it became the final selected submission.

2.6 Late-Stage Latent-Structure Models

After the accounting breakthrough, the larger working notebook explored more complex latent-structure models, including sparse entity-effect models and low-rank matrix factorization. These models attempted to capture residual school-assessment-subgroup patterns that were not fully explained by covariates or accounting identities. They are important for after-action reflection because some of them achieved stronger private leaderboard scores than the selected final model. However, they depended on a more complicated chain of experimental priors and residual targets that was not incorporated into the cleaned report notebook. For that reason, they are discussed later as evidence about model-selection tradeoffs, rather than described here as part of the selected final pipeline.

3 Final Selected Model

The final selected submission was `submission_final_accounting_solver_convex_te_blend_prior.csv`. This model had two main stages. First, it constructed a strong machine-learning prior from a convex blend of previously trained models. Second, it converted the target percentages into estimated counts of proficient students, and then applied subgroup accounting identities within each `SCHOOL × ASSESSMENT_NAME` group.

3.1 Stacked Machine-Learning Prior

Let q_i denote the prior prediction for row i before applying the accounting solver. The prior was a convex weighted average of six saved model predictions:

$$q_i = 0.624592p_{i,1} + 0.207687p_{i,2} + 0.127595p_{i,3} + 0.040027p_{i,4} + 0.000082p_{i,5} + 0.000017p_{i,6}.$$

The weights are nonnegative and sum to one, so q_i is an average of existing model predictions. This made the prior stable and easy to audit.

Table 3: Components of the blended prior.

Term	Model description	Weight	OOF MSE
$p_{i,1}$	LightGBM with target/statistical encoding	0.624592	82.908
$p_{i,2}$	LightGBM long run, 100k trees	0.207687	93.726
$p_{i,3}$	LightGBM long run, 80k trees	0.127595	94.163
$p_{i,4}$	ExtraTrees	0.040027	110.749
$p_{i,5}$	LightGBM selected 12k run	0.000082	98.779
$p_{i,6}$	Random Forest, 500 trees	0.000017	122.328

This stacked prior had an OOF MSE of 78.0715 before any accounting adjustment. Most of the weight went to the target/statistical-encoded LightGBM, but the long LightGBM runs and ExtraTrees still contributed enough different prediction behavior to improve the average.

3.2 Percentage-to-Count Conversion

The accounting model used the fact that each row contains both a proficiency percentage and a student count. For a known training row, the approximate number of proficient students was computed as

$$k_i = \text{round} \left(\frac{N_i y_i}{100} \right),$$

where y_i is the observed `PERCENT_PROFICIENT` and N_i is `N_STUDENTS`. For a test or validation row, y_i is unknown, so the blended prior gave the estimated proficient count

$$m_i = \frac{N_i q_i}{100}.$$

The solver was built in count space rather than percentage space. After solving for an adjusted count, the prediction was converted back to a percentage.

3.3 Subgroup Accounting Identities

For each group

$$g = (\text{SCHOOL}, \text{ASSESSMENT_NAME}),$$

the model used two subgroup identities:

$$k_{\text{All}} = k_{\text{Female}} + k_{\text{Male}},$$

$$k_{\text{All}} = k_{\text{ED}} + k_{\text{NotED}}.$$

Here, ED denotes **Economically Disadvantaged** and NotED denotes **Not Economically Disadvantaged**. These identities were applied only when the corresponding student counts were compatible. Specifically, an identity was used only if

$$|N_{\text{All}} - (N_a + N_b)| \leq \max(1, 0.02N_{\text{All}}).$$

This tolerance allowed for small reporting or rounding inconsistencies, but avoided forcing an accounting equation when the row counts clearly did not match.

3.4 Direct Algebraic Reconstruction as Motivation

Before the final solver, we tested a simpler direct reconstruction idea. If two related subgroup counts were known, the third could be inferred algebraically. For example,

$$\begin{aligned}\tilde{k}_{\text{All}} &= k_{\text{Female}} + k_{\text{Male}}, \\ \tilde{k}_{\text{Female}} &= k_{\text{All}} - k_{\text{Male}}.\end{aligned}$$

The reconstructed count was converted back to a percentage:

$$\tilde{y}_i = 100 \cdot \frac{\tilde{k}_i}{N_i}.$$

This direct reconstruction was not the final submitted model, but it motivated the final solver because it showed that subgroup identities contained very strong predictive information. The final model generalized this idea by solving all available subgroup equations in a group at once, instead of reconstructing one row at a time.

3.5 Constrained Accounting Solver

For each school-assessment group g , let k_g be the vector of unknown proficient-student counts and let m_g be the vector of prior counts from the stacked model. The accounting equations can be written as

$$B_g k_g + c_g = 0.$$

The matrix B_g contains coefficients from equations such as

$$k_{\text{All}} - k_{\text{Female}} - k_{\text{Male}} = 0,$$

$$k_{\text{All}} - k_{\text{ED}} - k_{\text{NotED}} = 0.$$

The vector c_g accounts for known training counts that appear in the same school-assessment group. The solver then estimated adjusted counts by minimizing

$$\hat{k}_g = \arg \min_{k_g} [\|k_g - m_g\|_2^2 + 100000 \|B_g k_g + c_g\|_2^2],$$

The first term keeps the solution close to the blended machine-learning prior. The second term penalizes violations of the subgroup accounting equations. The equation weight was selected from $[100, 1000, 10000, 100000]$, and the final selected value was 100000. This made the accounting equations behave almost like hard constraints whenever the student counts were compatible.

In the implementation, the bounded least-squares version was used so that solved counts could not fall below zero or exceed the number of students in the row.

3.6 Final Shrinkage Toward the Solver

The raw solver output was not used directly. Instead, the model moved partway from the blended prior toward the accounting-solver prediction. After converting the solved count back to a percentage,

$$\tilde{y}_i = 100 \cdot \frac{\hat{k}_i}{N_i},$$

the final prediction was

$$\hat{y}_i = \begin{cases} q_i + 0.932(\tilde{y}_i - q_i), & \text{if the accounting solver applies,} \\ q_i, & \text{otherwise.} \end{cases}$$

The shrinkage value 0.932 was selected by OOF performance. It means that when the solver produced a valid adjustment, the final prediction moved 93.2% of the way from the blended prior toward the accounting solution. Rows without a valid accounting equation kept the blended prior prediction. All final predictions were clipped to the feasible range $[0, 100]$.

3.7 Final Model Summary

Table 4: Final selected accounting-solver configuration.

Quantity	Value	convex	Meaning
Base prior	Six-model blend		Starting prediction q_i before accounting adjustment.
Prior OOF MSE	78.0715		Error before the accounting solver.
Equation weight	100000		Strength of the subgroup accounting equations.
Prior weight	1.0		Strength of the penalty for moving away from the prior count.
Solver shrinkage	0.932		Fraction of movement from prior prediction to solver prediction.
OOF adjusted rows	81,593		Training rows where a valid solver adjustment was available.
Test adjusted rows	45,105		Test rows where a valid solver adjustment was available.
Final OOF MSE	54.2930		Error after applying the accounting solver.
Public MSE	31.361		Public leaderboard score of the final selected submission.

The accounting correction reduced OOF MSE from 78.0715 to 54.2930. This showed that the final model’s main improvement did not come from adding another generic supervised learner. It came from converting predictions into count space and using the subgroup structure of the assessment records.

Results and Error Analysis

The final selected submission was the accounting-solver model with the convex target/statistical-encoding stacked prior. It achieved a public leaderboard MSE of 31.361 and a private leaderboard MSE of 30.533. The second selected submission was also an accounting-solver but with a different prior family and performed worse, thus was discarded from the discussion. However, several later matrix-factorization submissions, which were not selected, achieved stronger private leaderboard scores.

Table 5: Selected and stronger unselected leaderboard results.

Submission / model description	Public MSE	Private MSE	Selected
Final accounting solver with convex TE/blend prior	31.361	30.533	Yes
Accounting solver with top-3 LightGBM prior	31.408	30.573	Yes
Matrix factorization, rank 16, school regularization 500	29.334	27.921	No
Matrix factorization, rank 15, regularization 1000	29.262	27.944	No
Matrix factorization best OOF version	29.448	28.057	No

The selected model, therefore, performed well relative to the earlier supervised-learning baselines, but it was not the strongest model discovered.

Out-of-fold diagnostics show where the final accounting solver helped most. The convex blended prior had OOF MSE 78.0715 before accounting adjustment. After the solver, OOF MSE fell to 54.2930. This improvement was concentrated in rows where a valid subgroup accounting adjustment was available.

Table 6: OOF error decomposition for the final selected model.

Row group	Rows	Prior MSE	Final MSE
All OOF rows	144,921	78.07	54.29
Rows with accounting adjustment	81,593	66.00	23.77
Rows without accounting adjustment	63,328	93.62	93.62
Endpoint rows, $y = 0$ or $y = 100$	8,246	142.79	102.03
Interior rows, $0 < y < 100$	136,675	74.17	51.41

The main error pattern was therefore based on the coverage. When the subgroup accounting structure was applied, the solver reduced OOF MSE from 66.00 to 23.77. When it did not apply, the prediction remained equal to the blended prior, so there was no improvement. This explains why the accounting model produced such a large overall gain while still leaving room for later latent-structure models: the solver was extremely effective on rows with usable subgroup identities, but it could not directly improve rows outside those identities.

The second systematic error pattern was boundary behavior. Rows with true values at 0 or 100 remained harder than interior rows. Even after accounting adjustment, endpoint rows had OOF MSE 102.03, compared with 51.41 for interior rows. The model tended to overpredict very low outcomes and underpredict very high outcomes, which seems to be a shrinkage pattern. This was one reason later bounded-target transformations and matrix-factorization models were explored, although the final selected model remained the accounting-solver version.

Overall, the results show that the selected model’s strength came from correcting structurally linked subgroup rows, not from uniformly improving every row type. Its main weakness was that rows where the accounting identities correction could not be applied, still depended on the stacked prior. The stronger matrix-factorization models suggest that many of these remaining errors were due to latent school-assessment-subgroup interactions that were not fully captured by the accounting equations (not in `final_pipeline_code.ipynb` file, refer to `working_file.ipynb`).

Model-Driven Insights

Three main structures were crucial for the prediction project during our model development process: repeated group behavior, subgroup accounting relationships, and latent school-assessment interactions. The improvement of model performance came only when models could combine the patterns of the data, and the target was controlled by many overlapping effects.

The first major insight came from target/statistical encoding. For schools, districts, counties, assessments, subgroups, and their interactions, we added group summaries that were leakage-safe. This led to a large improvement over the base-feature tree models and showed that historical performance patterns attached to repeated entities were highly informative. However, we needed to be very cautious to construct these features out-of-fold since they depended on response variables. Otherwise, models could memorize training labels during the encoding, instead of learning patterns that generalized.

The second and most significant insight was the accounting structure included in the score table. Rows within the same `SCHOOL × ASSESSMENT_NAME` group were dependent because subgroup counts were related. The accounting solver led to a large improvement. It showed that much of the prediction problem could be treated differently. Instead of using ordinary regression, the problem could be reframed as a consistency problem based on related subgroup counts. In other words, the model not only made good predictions for each row separately, but also for the agreement with other rows from the same school and assessment.

The final insight came from the stronger matrix-factorization models, which were unselected. Their performance on the private leaderboard showed that even with target encoding and accounting constraints, there were still hidden patterns across schools, assessments, and subgroups that were not captured by our selected model. These latent structures couldn’t be explained by a singular variable. Low-rank matrix-factorization models were able to capture some of these hidden interaction patterns.

Conclusion: Discussion, Limitations, and Reflections

This project showed that the prediction task performed best when treated as a structured regression problem rather than a simple tabular modeling problem. Early linear, step-function, and tree-based models were very essential since they built benchmarks and showed that nonlinear interactions were important. Target/statistical encoding then showed that repeated patterns across school, district, assessment, and subgroup carried strong predictive information. The biggest and most significant improvement came when we recognized that subgroup rows were highly related through student-count relationships. These predictions could be adjusted in count space using accounting identities.

The final selected submission, `submission_final_accounting_solver_convex_te_blend_prior.csv`, combined a convex stacked machine-learning prior with a constrained accounting correction. It achieved a public leaderboard MSE of 31.361 and a private leaderboard MSE of 30.533. This model was selected because it was accurate, interpretable, easily reproducible, and directly related to the structure of the assessment records.

The main limitation of the selected model was that the accounting solver only improved rows where valid subgroup identities were available. Rows without these relationships still depended on the stacked prior. The solver also relied on approximate count relationships. Therefore, reporting inconsistencies, rounding, or mismatched subgroup definitions could reduce the accuracy. More generally, the selected model failed to fully capture hidden interactions among schools, assessments, and subgroups.

The main weakness of the workflow was in the final model selection. Later matrix-factorization submissions succeeded in achieving stronger scores on the private leaderboard. Thus, our modeling process did lead to better predictive structure than the final submission we selected. Looking back, the final selection process focused too much on explainability rather than on better predictive performance. The problem was not that better models were unavailable. Instead, our final submission decision was too conservative.

If we did the project again, we would like to balance better among interpretability, OOF performance, public leaderboard results, and model diversity. We would also include the best hidden/latent-structure models in the cleaned final pipeline earlier. We would like to combine matrix-factorization predictions with the accounting solver. The matrix-factorization model could provide an additional prior, while the accounting solver would maintain subgroup consistency. This would combine interpretability with the predictive power of hidden school-assessment patterns.

Disclosures

For the purposes of this project, we used ChatGPT for coding implementation assistance and idea generation (this was due to earlier ambiguity in the assignment wording, which landed in our favor). Furthermore, ChatGPT was also used only for finding LaTeX code for tables so that we can easily put our tables in LaTeX format for better visuals in our report. The report material was not generated with LLM assistance; we referred to our coding blocks and workflow in order to construct our report wording. We did not generate this report via LLM since that would be a violation. We used LaTeX. We primarily used the visual editor in LaTeX Overleaf to generate this document.

Appendix

See the attached zip file and make sure to read the README.md file to understand how to read and run code.

There are 3 notebooks, only one is needed, namely final_model_only.ipynb. This can generate the final submission and only creates the models needed for that.

To understand the purpose of the other notebooks, refer to README.md.